



24BCF10059 - Lakshdeep

Experiment 6

Aim :

To study and implement advanced SQL techniques such as conditional/custom ordering using CASE expressions inside ORDER BY, and self-joins for modelling hierarchical (manager–employee) relationships.

Question 1

A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges.

Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day. The difference should be calculated as:

Difference = Apples Sold - Oranges Sold

Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.

Solution :

```
SELECT sale_date , SUM(  
    CASE  
        WHEN fruit='apples' THEN sold_num  
        ELSE -sold_num  
    END  
) AS Diff  
FROM Sales  
GROUP BY sale_date  
ORDER BY sale_date;
```

Question 2

A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where:

- `id` represents the current node.
- `p_id` represents the parent node.

Classify every node into one of the following categories:

- **Root:** A node that does not have any parent.
- **Leaf:** A node that has a parent but does not have any children.
- **Inner:** A node that has both a parent and at least one child.

Display the node ID along with its category. Return the result sorted by node ID.

Solution :

```
SELECT
  id,
  CASE
    WHEN p_id IS NULL THEN 'Root'
    WHEN id IN (SELECT p_id FROM Tree WHERE p_id IS NOT NULL) THEN 'Inner'
    ELSE 'Leaf'
  END AS type
FROM Tree
ORDER BY id;
```

Learning Outcomes :

After completing this experiment, the following learning outcomes were achieved:

- Understood how to use **CASE expressions with aggregate functions** to perform conditional calculations.
- Learned to calculate the **difference between two categories of sales** using `SUM()` and conditional CASE statements.
- Applied **GROUP BY** to generate date-wise sales comparison reports.
- Learned to use **ORDER BY** for sorting the final result based on the sale date.
- Understood how to classify hierarchical nodes as **Root, Inner, or Leaf** using CASE expressions.
- Learned to identify parent–child relationships using **subqueries** within SQL conditions.
- Applied **IS NULL** to identify root nodes that do not have a parent.
- Strengthened practical understanding of **conditional logic, aggregation, subqueries, and sorting** in SQL.